

Deep Generative Models

Chapter 3: Generation by Explicit Distribution Learning

Ali Bereyhi

`ali.bereyhi@utoronto.ca`

Department of Electrical and Computer Engineering
University of Toronto

Summer 2026

Need for a Computationally-Feasible Model

Clearly, modeling data distribution as a whole is computationally infeasible

- ! The model needs to cover an exponentially large data-space
- ! Direct sampling could be exponentially hard

To handle these issues, various modeling approaches can be deployed

- 1 autoregressive (AR) modeling ←
- 2 energy-based modeling
- 3 flow-based modeling

In this section, we look into AR modeling

AR Modeling

Using the probability **chain-rule**: we can **expand** the **data distribution** as

$$\begin{aligned} P(x) &= P(x_1, \dots, x_d) \\ &= P(x_1) P(x_2|x_1) P(x_3|x_1, x_2) \dots P(x_d|x_1, \dots, x_{d-1}) \\ &\quad \begin{array}{ccccccc} & \uparrow & \uparrow & \uparrow & & \uparrow & \\ & x_{<1} & x_{<2} & x_{<3} & & x_{<d} & \end{array} \\ &= P(x_1|x_{<1}) P(x_2|x_{<2}) P(x_3|x_{<3}) \dots P(x_d|x_{<d}) \\ &= \prod_i P(x_i|x_{<i}) \end{aligned}$$

Key Point in AR Modeling

d-dimensional object** is decomposed into product of **d 1-dimensional objects

AR Model: Generic

AR Model: Generic Form

AR models represent the *data distribution* as

$$\prod_i P_{\mathbf{w}_i}(x_i | x_{<i})$$

for some *computational models* $P_{\mathbf{w}_1}, \dots, P_{\mathbf{w}_d}$

- + It does not seem to be any *inaccuracy* in this modeling! Right?!
- *Exactly!* If each conditional distribution $P(x_i | x_{<i})$ is *learned perfectly*, the model is *exact!*
- + The AR model reminds me of *LMs!*
- Exactly! *LMs* are indeed AR models
- + Can we generate any *other modality* using *AR modeling*?
- Sure!

AR Modeling: *Example of MNIST*



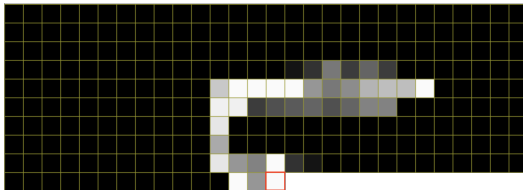
$$P_{\mathbf{w}}(x_1)$$

AR Modeling: *Example of MNIST*



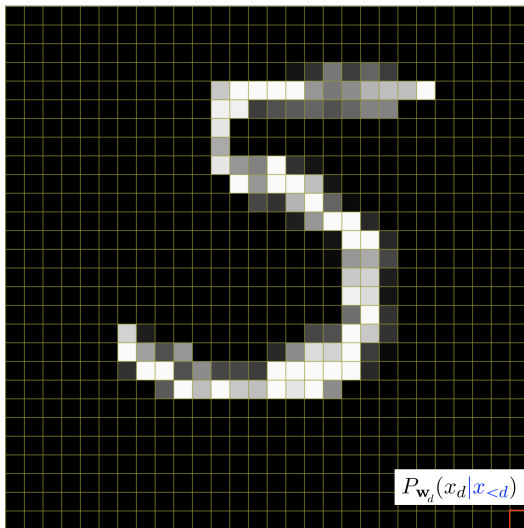
$$P_{\mathbf{w}_2}(x_2|x_1)$$

AR Modeling: *Example of MNIST*



$$P_{\mathbf{w}_i}(x_i | x_{<i})$$

AR Modeling: *Example of MNIST*



Sampling AR Models

Say we have trained the AR model

$$\prod_i P_{\mathbf{w}_i} (x_i | x_{<i})$$

to **sample** this model, we

- **first sample** $x_1 \sim P_{\mathbf{w}_1} (x_1) \rightsquigarrow \text{complexity} = \mathcal{O}(C)$
- then **use** x_1 to **sample** $x_2 \sim P_{\mathbf{w}_2} (x_2 | x_1) \rightsquigarrow \text{complexity} = \mathcal{O}(C)$
- $\vdots$
- finally **use** $x_{<d}$ to **sample** $x_d \sim P_{\mathbf{w}_d} (x_d | x_{<d}) \rightsquigarrow \text{complexity} = \mathcal{O}(C)$

Conclusion: **Linear Sampling Complexity**

We can sample with **linear** complexity, i.e., $\mathcal{O}(Cd)$!

Sampling AR Models: Trade-off

Conclusion: Linear Sampling Complexity

We can sample with linear complexity, i.e., $\mathcal{O}(Cd)$!

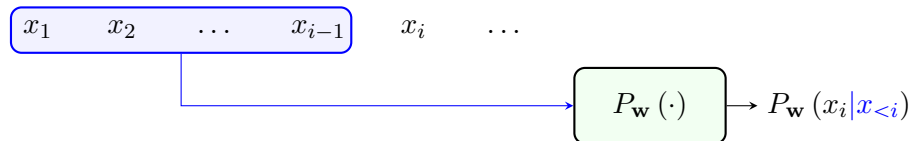
- + Sounds weird! Isn't there **any cost** we pay for this **huge complexity reduction**?
 - Sure! The cost is **autoregressive** generation which is **slow**!
-
- In a joint model $P_{\mathbf{w}}(x)$, we generate the sample in **one-shot**
 - ↳ For example, the **whole image at once**
 - In AR models, we generate the sample **entry-by-entry**
 - ↳ For example, we generate the image **pixel-by-pixel**

AR Model with Shared Parameters

AR Model with Shared Parameters

In practice, all conditional distributions are represented by the *same* model

$$P_{\mathbf{w}}(x) = \prod_i P_{\mathbf{w}}(x_i | x_{<i})$$



Probability Computation via Context Extraction

Computational AR models follow the same approach as we did in **LMs**

- For each x_i , a **context** is extracted from $x_{<i}$, i.e.,

$$\mathbf{c}_i = f_{\boldsymbol{\theta}}(x_{<i})$$

where $\boldsymbol{\theta} \subset \mathbf{w}$ contain a major subset of **model parameters**

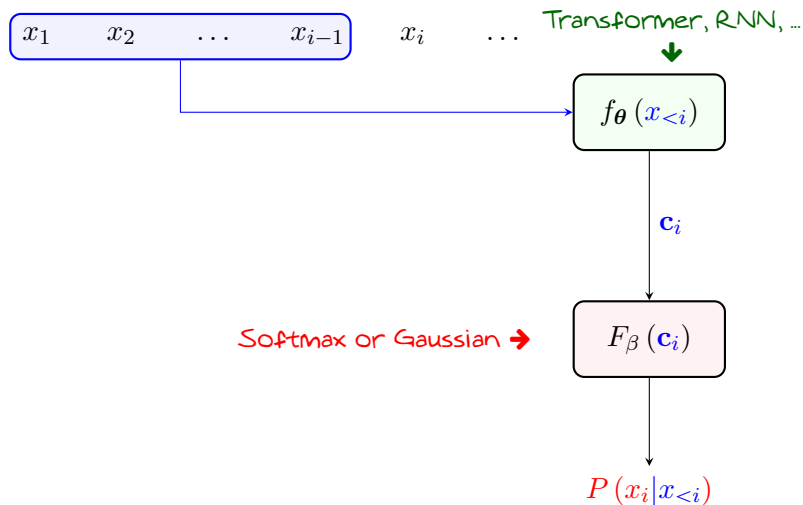
- We compute $P(x_i|x_{<i})$ from **the context** using a **probability-defining layer**

$$P_{\mathbf{w}}(x_i|x_{<i}) = F_{\beta}(\mathbf{c}_i) = F_{\beta}(f_{\boldsymbol{\theta}}(x_{<i}))$$

where $\mathbf{w} = \{\boldsymbol{\theta}, \beta\}$; examples of such layers are

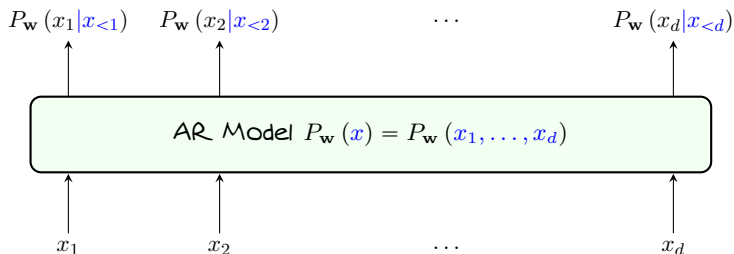
- ↳ a **Soft_{max} activated layer**, or
- ↳ a **Gaussian distribution** whose mean and covariance depend on $\mathbf{c}_i$

Probability Computation via Context Extraction



AR as Computational Model

From a computational perspective: AR is a model with d inputs and d outputs



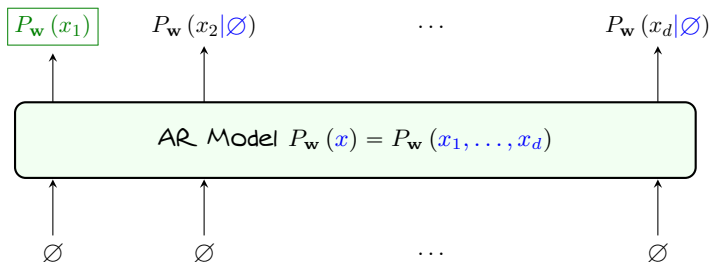
This computational model can be realized by *any architecture*

- Basic MLP or an RNN ✓
- Transformer ✓
- Convolutional network ✗

Autoregressive Generation

- + Say we trained the model, how do we generate a sample?
- We should do it **autoregressive!**

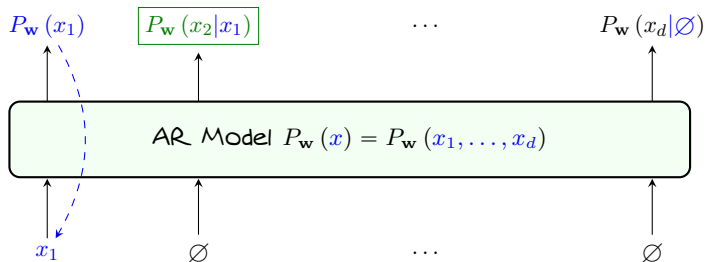
We first sample the *first component*



Autoregressive Generation

- + Say we trained the model, how do we generate a sample?
- We should do it **autoregressive!**

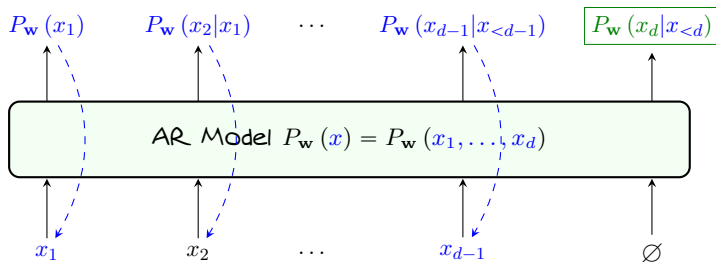
We then inject *first component* and sample *the second component*



Autoregressive Generation

- + Say we trained the model, how do we generate a sample?
- We should do it **autoregressive!**

We keep repeating till we sample *the last component*



Bottleneck: *Slow Generation*

- + This sounds to take a *long time!*
- Sure! And this is a key *challenge* of *autoregressive generation*

Example: Say we want to generate samples from *CelebA*

↳ *CelebA* is a dataset of 200K celebrity face images

↳ Each image is RGB with resolution 178×218

Say each $x_i \in \mathbb{R}^3$ is an RGB pixel $\rightsquigarrow$ we need *39K forward passes!*

Moral of Story

AR models are *slow* in generation due to their *autoregressive* nature of *sampling*

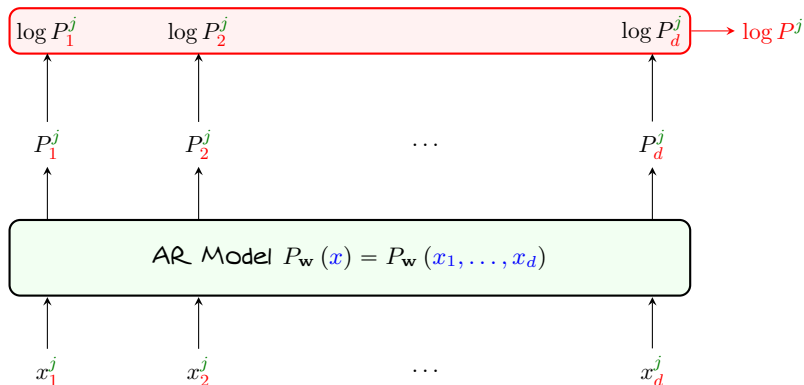
Training AR Models via MLE

- + What about their training?
- Well, we should follow the general recipe $\rightsquigarrow$ MLE

Say we have $\mathbb{D} = \{x^j \text{ for } j = 1, \dots, n\}$: the log-likelihood is computed as

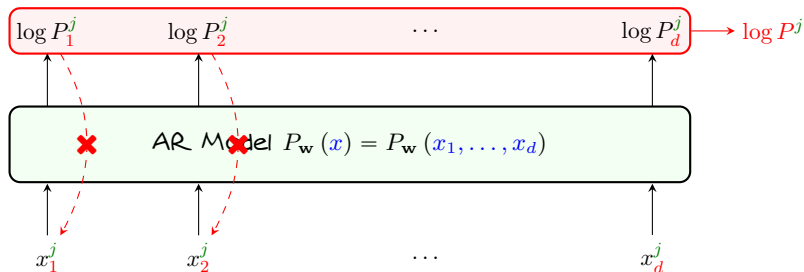
$$\begin{aligned} \frac{1}{n} \sum_{j=1}^n \log P_{\mathbf{w}}(x^j) &= \frac{1}{n} \sum_{j=1}^n \log \prod_i P_{\mathbf{w}}(x_i^j | x_{<i}^j) \\ &= \frac{1}{n} \sum_{j=1}^n \sum_{i=1}^d \log P_{\mathbf{w}}(x_i^j | x_{<i}^j) \end{aligned}$$

Training AR Models via MLE



- + This does not seem to be **autoregressive** anymore?
- If model does **parallel computation**, e.g., as in transformers, **No!**
 - ↳ This is the **teacher-forcing** training!

Teacher-Forcing Training



Teacher-Forcing Training

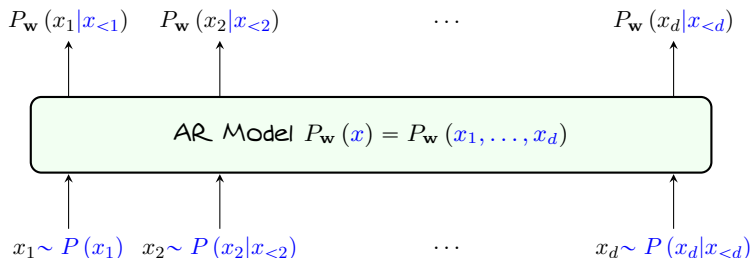
In AR models, likelihood computation can be in parallel, i.e., non-autoregressive, using the **ground truth**

- ↳ This requires **parallelized** computational model, e.g., transformers
- ↳ This looks like **a teacher forcing input** to be sampled from **data distribution**

Teacher-Forcing: *Exposure Bias*

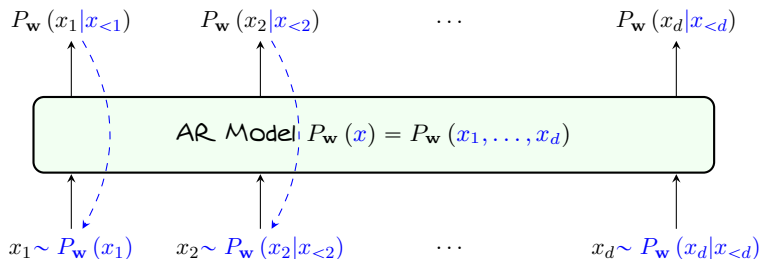
Teacher forcing makes difference between training and generation

During training the input is sampled from *data distribution*



Teacher-Forcing: *Exposure Bias*

However, during generation inputs are sampled from *model*



This means that during *generation*, model sees samples of *different* distribution

- + Don't we learn the data distribution?!
- At the end of the day, we are still *far away from it!*
 - ↳ We really *cannot* learn *exact* distribution! We get close to it

Teacher-Forcing: *Exposure Bias*

Exposure Bias

Exposure bias occurs due to mismatch between input distributions AR model sees during training and generation

Example: For instance in LMs, exposure bias can cause such a scenario

📖 the cat jumps on a **mat** and goes to sleep

➡ the cat jumps on a ...

⚙ ... **dog** barks loudly

- + *Is there any solution to that?!*
- Yes, there are several solutions
 - ↳ **Scheduled sampling** gradually replaces **true samples** with **generated ones**
 - ↳ **Reinforcement learning** can be also used to learn a **reward system** that improves **model generalization**

Wrapping Up: AR Models

- AR models represent the distribution as *product of conditionals*
- Sampling these models is *very easy*
 - ↳ It is however *very slow*
- Training can *speed up* using teacher forcing approach
 - ↳ It can however lead to *exposure bias*

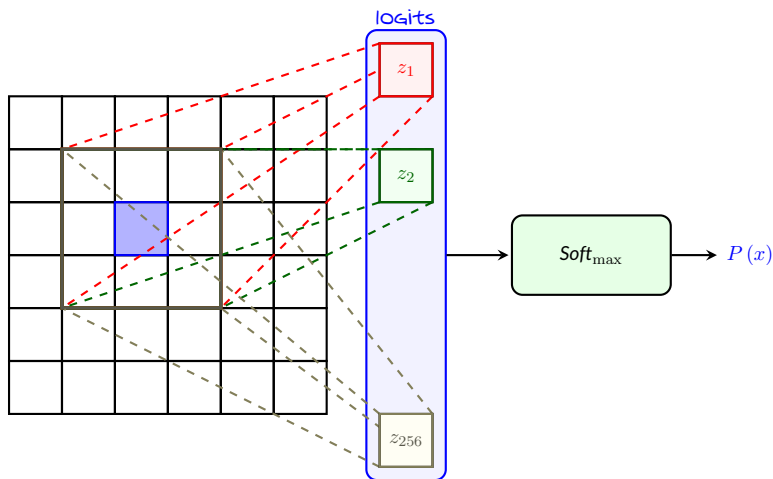
There are various examples of AR models

- ✓ Language models
- ➔ RNN and CNN based AR models for *image generation*
 - ↳ PixelRNN and PixelCNN
- ➔ Transformer-based AR models for *image generation*
 - ↳ Image GPT

We next take a look at *visual* AR models

Spatial Context via Convolution

We can compute a *distribution* for a *single pixel* using multiple *kernels*

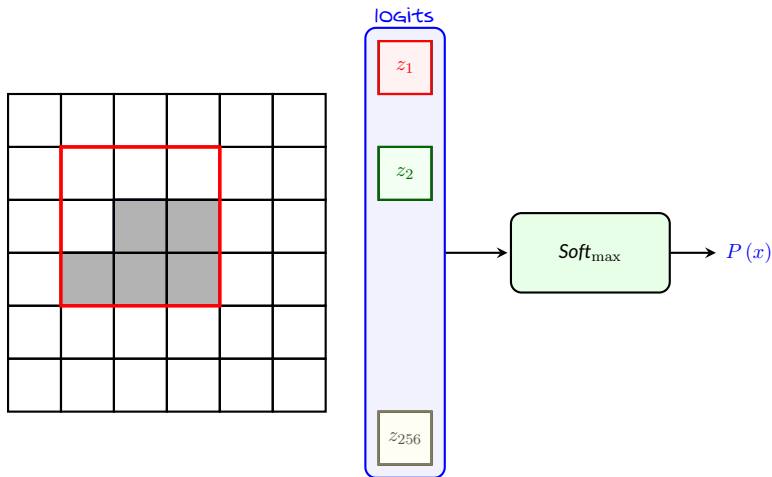


Valid Conditional Distribution

- + Wait a moment! The *conditional* distribution however depends on *all pixels* in the *receptive field*! Including the *pixel itself*! Right?!
- *Yes!* You are right!
- + Then, this *cannot* be a *valid conditional distribution*!
 - We need to compute *distribution conditioned* to *what we had before*!
 - This distribution depends on the *pixel* itself and *some future ones*!
- We can resolve this using *masked filter*
 - We did the same in *Transformers*!

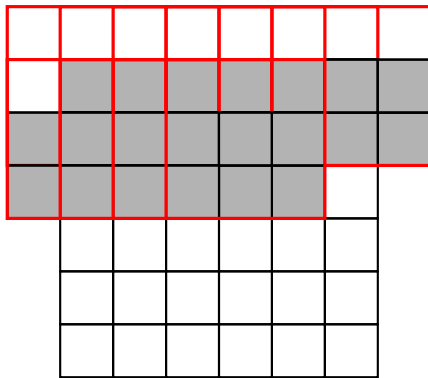
Masking Kernels

We mask kernels at the *pixel position* and the *future positions*

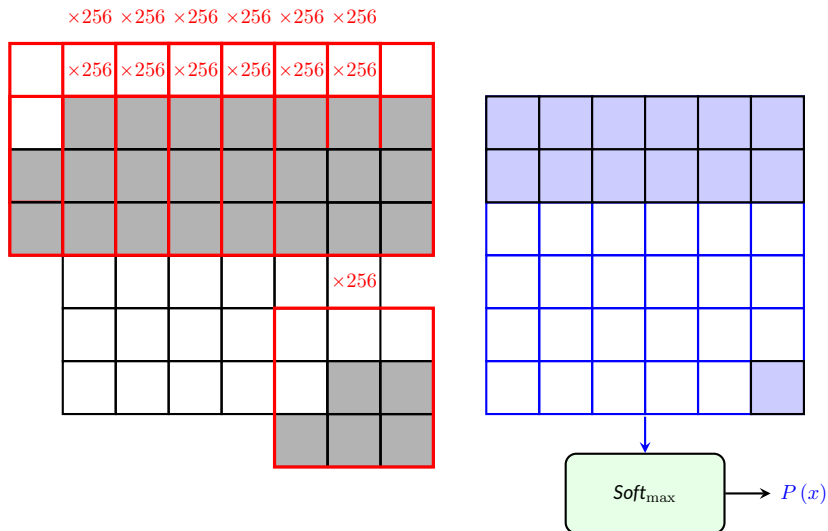


Masked Convolution

We can hence compute *valid context* via *masked convolution*



Masked Convolution: *Valid Conditional Distribution*



Short Range Conditioning

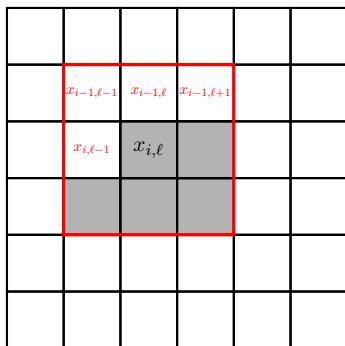
Short Range Impact of Masked Convolution

The computed *conditional distributions* are *valid*; however, they are conditioned only on nearby pixels within the *receptive field of kernel*

In terms of AR modeling, we assume

$$P(x_{i,\ell} | x_{<i,\ell}) \approx P(x_{i,\ell} | x_{\text{neighbors}})$$

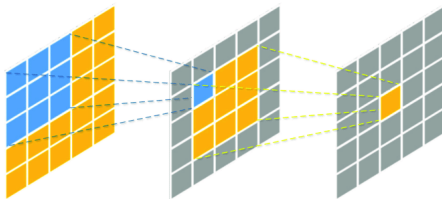
which *weakens spatial correlation capturing*



Recap: Receptive Field of Deep CNN

Recap: Deep Convolution

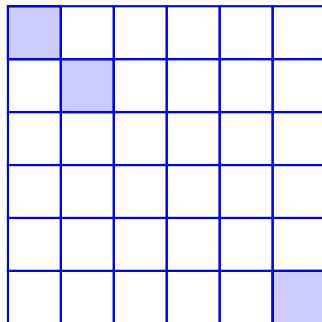
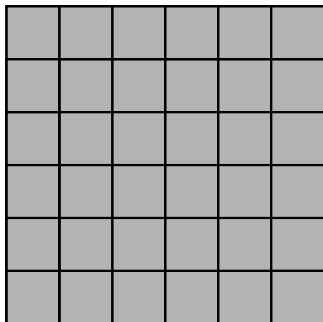
Recall as we go *deeper* in CNN $\rightsquigarrow$ *effective receptive field expands*



Solution: Deep Masked Convolution

Using *deep masked* CNN $\rightsquigarrow$ *effective receptive field covers all previous pixels*

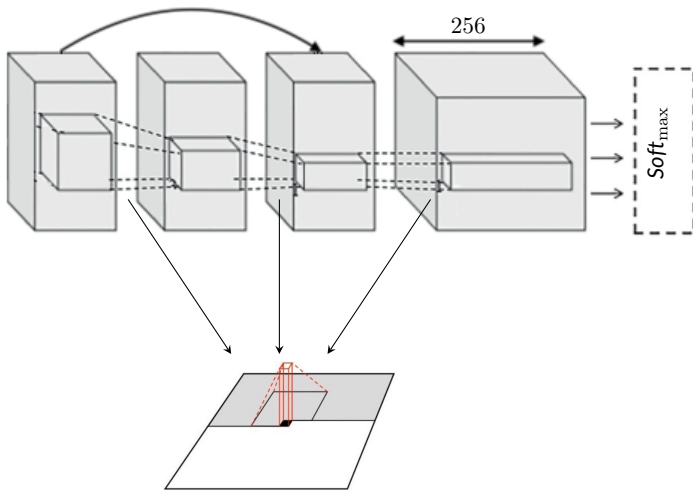
Deep Masked Convolution



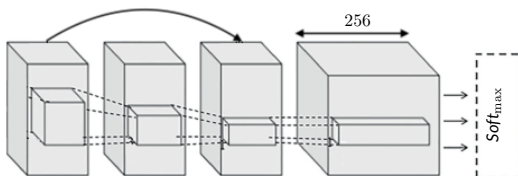
As we go deep with *masked* convolutional layers

- *receptive field* expands to the *whole image scene*
- since all kernels are *masked*, the final *receptive field* is also *masked*
- this extracts an *autoregressive context* that we need

PixelCNN: AR Model with Deep Masked Convolution



PixelCNN: Few Notes



In practical implementation of PixelCNN

- Typically the network is **quite deep** > 15
 - ↳ To guarantee **accurate conditioning**
- Deep networks can quickly experience **vanishing gradients**
 - ↳ We use **skip connections** and **gating** to reduce **vanishing gradients**
- Final layer extracts **256 features for each pixel**
 - ↳ We typically condition RGB channels **right after each other**

PixelCNN: Training

We can train PixelCNN via **MLE**

```
Train_PixelCNN(D:dataset):  
1: for multiple epochs do  
2:   Sample a batch of  $n$  images  $\{x^j : j = 1, \dots, n\}$   
3:   for  $j = 1 : n$  do  
4:      $\mathbf{p}_{1:d}^j \leftarrow \text{MaskedCNN}(x^j)$   
5:   end for  
6:   Update model using  $\text{Opt\_avg} \left\{ \sum_i \nabla \log \mathbf{p}_i^j [x_i^j] \right\}$   
7: end for  
8: return trained model
```

Parallel Processing

Using **masked** convolution, we can process all pixels in **parallel** while we **train**

PixelCNN: Generation

Generation is still *autoregressive*

```
Sample_PixelCNN( ):
```

```
1: Set  $x^{(0)} = \emptyset$  #prior sampling
2: for  $i = 1, \dots, d$  do
3:    $\mathbf{p}_{1:d} \leftarrow \text{MaskedCNN}(x^{(i-1)})$ 
4:    $x_i \leftarrow \text{multinomial}(\mathbf{p}_i, \text{\#smpl}=1)$ 
5:   Update  $x^{(i)} \leftarrow \text{Replace } x_i \text{ in } x^{(i-1)}$ 
6: end for
7: return  $x^{(d)}$ 
```

Wrap Up and Final Notes

AR modeling gives an efficient way to **computationally** model **data distribution**

↳ It deploys **autoregressive sampling** leading from **slow generation**

We can realize them by

- **Recurrent** networks, e.g., LSTM
 - ↳ It explicitly **conditions** distribution through **model state**
 - ↳ We do **not** benefit from **parallel processing**
- **Masked** networks, e.g., **masked CNN or Transformers**
 - ↳ It **conditions** causally through **masking**
 - ↳ We enjoy the **luxury** of **parallel processing**

AR modeling are widely considered **slow** of **visual modalities**

↳ They are though getting **revisited** due to some **recent advancement**

RNN-based AR Models

Recurrent AR models are one of **primary computational** AR models: we have already seen the **recurrent LMs**

In these models, we use RNN to build conditional distribution

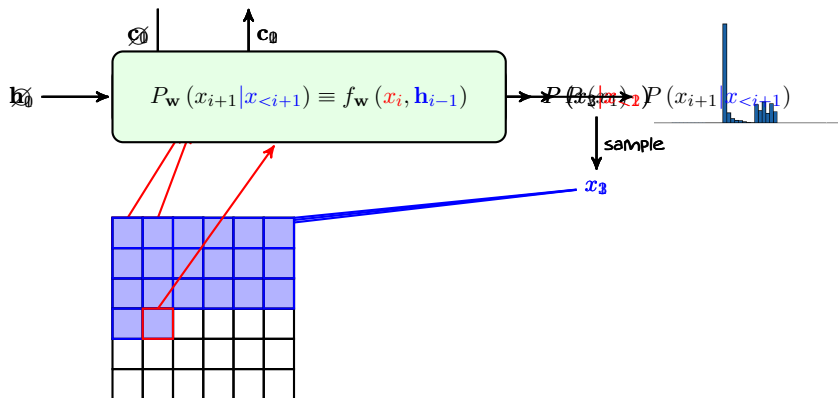
- 1 The model samples x_1 from a **learned prior**
- 2 Given x_i and the **state** of RNN at $i - 1$, the model
 - ↳ updates its **state** to represent **the context at i**
 - ↳ computes **Softmax**-activated output of context to estimate $P(x_{i+1} | x_{<i+1})$
- 3 The model iterates till d entries are over

Key Observation

We **cannot** do **parallel processing** $\rightsquigarrow$ **training** is also **slow**

PixelRNN: Visual Recurrent AR Model

Proposed in 2016, *PixelRNN* uses *LSTM* to extract context



PixelRNN: Training

We can readily train this model via **MLE**

```
Train_PixelRNN(D:dataset):
```

```
1: for multiple epochs do  
2:   Sample a batch of  $n$  images  $\{x^j : j = 1, \dots, n\}$   
3:   for  $j = 1 : n$  do  
4:      $\mathbf{p}_1^j, \mathbf{c}, \mathbf{h} \leftarrow \text{LSTM}(\emptyset)$            #define  $\emptyset$  to be start pixel  
5:     for  $i = 1, \dots, d - 1$  do  
6:        $\mathbf{p}_{i+1}^j, \mathbf{c}, \mathbf{h} \leftarrow \text{LSTM}(x_i^j, \mathbf{c}, \mathbf{h})$   
7:     end for  
8:   end for  
9:   Update model using  $\text{Opt\_avg} \left\{ \sum_i \nabla \log \mathbf{p}_i^j [x_i^j] \right\}$   
10: end for  
11: return trained model
```

PixelRNN: Generation

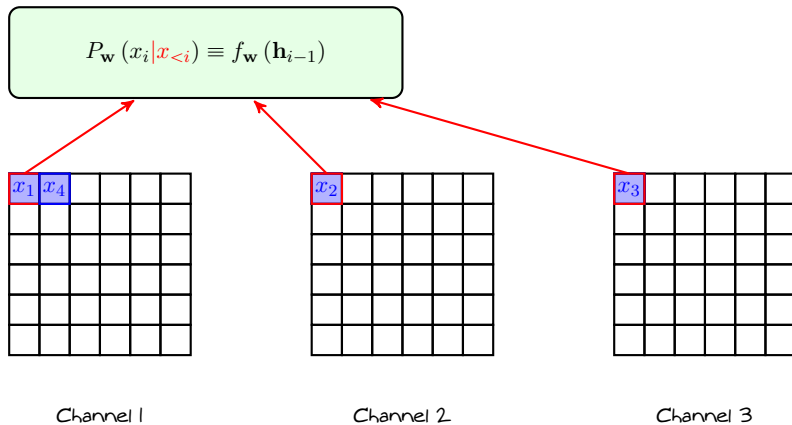
It's good to think how we can *generate* with this model

```
Sample_PixelRNN( ):
1:  $\mathbf{p}_1, \mathbf{c}, \mathbf{h} \leftarrow \text{LSTM}(\emptyset)$ 
2:  $x_1 \leftarrow \text{multinomial}(\mathbf{p}_1, \text{\#smpl}=1)$  #sample first pixel
3: for  $i = 1, \dots, d - 1$  do
4:    $\mathbf{p}_{i+1}, \mathbf{c}, \mathbf{h} \leftarrow \text{LSTM}(x_i, \mathbf{c}, \mathbf{h})$ 
5:    $x_{i+1} \leftarrow \text{multinomial}(\mathbf{p}_{i+1}, \text{\#smpl}=1)$  #AR generation
6: end for
7: return  $x = \text{DataType} \{x_1, \dots, x_d\}$ 
```

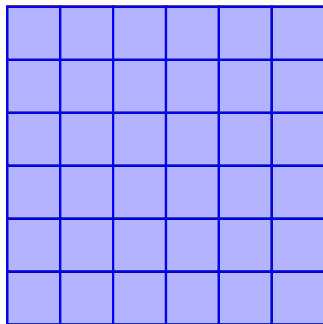
PixelRNN: Multi-channel

For multi-channel data, we typically *first* process *in depth*

↳ We expect that RGB pixels are *heavily correlated*



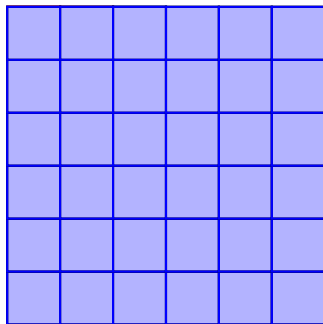
PixelRNN: Row Processing



Basic PixelRNN goes through images *row-wise* $\rightsquigarrow$ processing time $\propto d^2$

- For *generation* we need to complete these d^2 steps
- For *training* we can treat each row *independently* $\approx$ truncated BPTT
 - $\hookrightarrow$ *Faster* at the price of capturing *less spatial* correlation

PixelRNN: *Diagonal Processing*



Alternatively, we can process each pixel conditioned on its **top and left pixels**

- We can process **multiple pixels** in **parallel**
- Total **processing time** for a single image is $2d - 1$ or $\propto d$
 - ↳ **Fast** with **less loss** in capturing **spatial correlation**

PixelRNN: *Diagonal Processing* - Reduced AR Modeling

x_1	x_2				
x_7					

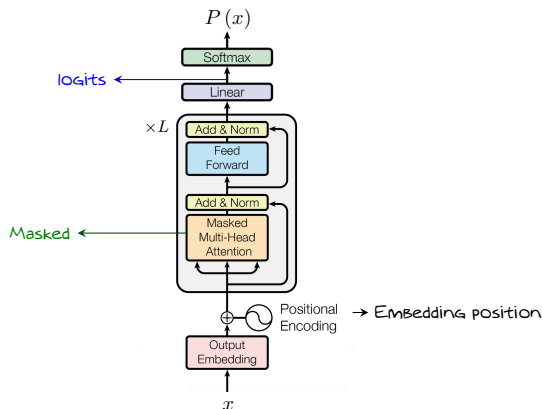
Attention

Note that we get the benefit of *parallel processing* due to *reduced* AR modeling

$$P(x_2|x_1) P(x_7|x_1, x_2) \approx P(x_7|x_1) P(x_7|x_1)$$

Transformer-Based AR

We can use the same approach as in LMs to extract **context** via **transformers**



This is **exactly** LM with **time** $\equiv$ **position** and **tokens** $\equiv$ **pixels**

ImageGPT: Extending LMs to Generate Pixels

A good example is **ImageGPT**:

it interprets **pixels** as **tokens** $\rightsquigarrow$ **image** is a **long sequence of tokens**

- **Vocabulary** has only 256 tokens
 - ↳ We can **patch multiple pixels** together to make **larger vocabulary**
 - ↳ With **patch-pixels**, we can use **tokenization** to reduce **vocabulary size**
- Each **pixel-patch** is embedded with an **embedding layer**
 - ↳ This embedding can be in general **smaller** than **vocabulary size**
 - ↳ It will give us a **numerical representation** of the **image patches**
- As we use **self-attention**, the model learns **spatial correlation** by itself
 - ↳ This is **more robust** as compared to **deep masked convolution**
 - ↳ It however needs **more data** to **start capturing** spatial pattern

Take a look at **ImageGPT Paper**